Response to Reviewers

IoT-66559-2026 (R1)

submitted as: FedKAN-IDS: Heterogeneity-Robust Federated Kolmogorov–Arnold Networks for NetFlow-Based IoT Intrusion Detection

revised to: Tuning Confounds the Comparison: What a Controlled Study of Federated KANs for NetFlow Intrusion Detection Actually Shows

We thank both reviewers for reports that were specific enough to act on. Two of the requested experiments changed our conclusions rather than confirming them, and we report that plainly below: Reviewer 2's comment 7 in particular led us to a result that weakens the manuscript's original headline claim. We have rewritten the paper around what the new measurements actually support.

Summary of what changed.

- Every architecture is now tuned over a learning-rate grid before being compared (1,200 runs for the sweep, 1,500 in total), and all headline numbers are re-reported under that protocol. The title, abstract and contributions changed as a result: the accuracy and variance claims of the first version are withdrawn, and the paper now reports a robustness-versus-compute trade-off.
- All experiments were re-run on a single machine so that no number in the paper is compared across hardware.
- Accuracy is now summarised by the mean over the final five rounds rather than the last round alone.
- Precision, recall, MCC and false-positive rate are reported for every cell.
- Communication cost is reported as bytes-to-target and rounds-to-target, not only total bytes after a fixed round budget.

Reviewer 1

R1.1 Novelty clarification: "The contribution is primarily empirical rather than methodological": articulate scientific insights beyond empirical KAN vs. MLP comparisons.

Response. We agree that "we compared two architectures" is not by itself a scientific insight, and in revising we found that the insight the study actually supports is a different and sharper one than the submitted version claimed. Our learning-rate sweep (see R2.7) shows that the accuracy and variance advantages attributed to KANs in the FedKAN literature, including in our own submitted manuscript, are confounded with *how much hyper-parameter tuning each architecture received*. Once each architecture is given its own best learning rate, the mean advantage and the variance advantage both largely disappear on the cell that produced our headline number. What

survives is a distinct and practically relevant property: the KAN reaches within a fraction of a point of its best accuracy across a wide band of learning rates, whereas the parameter-matched MLP requires tuning to be competitive at all. For federated deployment across IoT gateways, where per-client hyper-parameter search is not available, insensitivity to the learning rate is arguably the more useful property, and it is a claim about *optimisation behaviour* rather than about representational capacity. The revised paper is built around that claim, and the title and abstract have changed accordingly.

Where in the manuscript:

R1.2 Expand baselines: add comparisons with FedProx, SCAFFOLD, MOON, FedNova, or FedDyn beyond FedAvg-based MLPs.

Response. Added, on the cell that produced our headline number, ten seeds each. FedProx is the one that matters and it works against our original claim: it lifts the parameter-matched MLP from 88.72% to 94.92%, a gain of 6.19 pp, and cuts its seed variance from 14.21 to 2.67 pp, while lifting FedKAN-8 by only 0.44 pp. Under FedProx the two architectures are level and the sign in fact reverses: -0.26 pp in the KAN's disfavour ($p = 0.82$). Our submitted comparison therefore under-served the baseline in two independent ways, a shared learning rate and a FedAvg-only aggregator, and correcting either one removes most of the gap. MOON gives no benefit to either architecture (91.17% and 91.14%, $p = 0.32$ and $p = 0.55$).

We should add that this table was originally measured at exactly the shared step size the rest of the revision argues against, which would have been a fair objection. We therefore re-ran FedProx with each architecture at its own best rate: FedKAN-8 reaches 93.85% and the MLP 95.72%, a difference of -1.87 pp. The two corrections compound rather than cancel, and the revised Table **??** reports all three configurations side by side.

We must be candid about SCAFFOLD. Our implementation does not converge here: 49.80% for FedKAN-8, which is chance on a binary task, and 73.32% for the MLP. SCAFFOLD's control variates are derived for local SGD, and every experiment in this paper uses Adam; we believe that mismatch is the cause but we have not proved it. We therefore report the runs and explicitly draw *no* conclusion about SCAFFOLD from them, rather than present a number we suspect is our own error as evidence about the method. FedNova and FedDyn were not run; we prefer to say so than to add them hastily.

Where in the manuscript:

R1.3 IDS metrics: include Precision, Recall, F1-score, MCC, ROC-AUC, or False Positive Rate for comprehensive evaluation.

Response. Precision, recall and per-class F1 were already computed for every run; we have added them to the results tables together with MCC and the false-positive rate, which we derive exactly from the stored per-class recalls and accuracies. We verify the derivation rather than assert it: reconstructing each confusion matrix and recomputing precision reproduces the stored value for 163/163 binary runs. We do *not* report ROC-AUC, and we prefer to say so explicitly rather than pass over the request: our runs store predicted labels, not continuous scores, so ROC-AUC cannot be recovered from the released artefacts.

Where in the manuscript:

R1.4 Communication costs: beyond parameter transmission, add transmitted bytes, communication rounds, convergence efficiency, or training time.

Response. We now report bytes and rounds *to reach a fixed accuracy target* rather than total bytes after a fixed round budget, because the latter assumes both architectures must run the full schedule. Table **??** reports uplink-to-target together with the number of seeds that reach the target at all, which is the column a mean would hide: at the 90% target FedKAN-8 arrives on nine seeds of ten against seven for the parameter-matched MLP, while costing more bytes to get there. Training time and per-round wall-clock are included, and inference cost is answered under R1.5.

Where in the manuscript:

R1.5 Deployment claims: support Raspberry Pi statements with experimental evidence or present them more cautiously as potential future work.

Response. We have removed the Raspberry Pi statements rather than soften them, because we do not have that hardware and a number we cannot measure should not appear. In their place we report what we can measure: single-thread CPU inference with batch size 1, which is the closest deployment-like regime available to us. At matched parameters (3,280 against 3,362) FedKAN-8 costs 0.0995 ms per sample against 0.0050 ms for the parameter-matched MLP, a factor of 19.9, and 1.690 ms against 0.045 ms for a batch of 1000, a factor of 37.9. So that a reader can convert these to their own gateway we also report an anchor on the same machine: a 256×256 matrix multiply takes 0.024 ms. The full protocol, including the anchor, is Table **??** in the revised Section **??**. Parameter parity is not cost parity, and this now appears in the abstract.

Where in the manuscript:

R1.6 Generalization: present class-distribution skew and FedKAN performance as an empirical observation rather than a universally established property.

Response. We have removed the generalising language. Every statement linking class-distribution skew to FedKAN performance is now scoped explicitly to the three NetFlow-v2 datasets and the partitioning regimes we ran, and is presented as an empirical observation on that sample rather than an established property. We have also removed the sentences in the introduction and conclusion that projected the pattern onto unseen datasets. Reviewer 1 is right that three datasets cannot establish a monotone relationship, and our revised analysis (R2.7) gives a further reason for caution: the direction of the effect is sensitive to a hyper-parameter choice we had held fixed.

Where in the manuscript:

R1.7 Presentation: increase font sizes, improve confidence interval visibility, simplify long sentences.

Response. All figures have been redrawn: axis and tick label sizes are increased, confidence intervals are drawn as shaded bands with explicit whisker caps at the mean, and the colour palette is now colour-blind safe and locked per method across every figure so that one architecture

keeps one colour throughout. We have also gone through the manuscript for overlong sentences. We note one specific defect the reviewer's comment led us to: a sentence in the conclusion had lost its punctuation in the submitted PDF and read as "a reproducible failure mode seed 17's Dirichlet partition drives…". That sentence has been repaired.

Where in the manuscript:

Reviewer 2

R2.1 Theoretical assumptions: Theorem 1 assumes L-smoothness and μ-strong convexity, which cross-entropy loss with deep networks does not satisfy globally.

Response. The reviewer is correct, and on re-examining the section we found a second and more serious problem that we would rather state ourselves than have discovered later.

First, on the assumptions as raised: cross-entropy loss composed with either a KAN or an MLP is not globally L-smooth or μ-strongly convex, so Theorem **??** does not describe our training runs globally. We now state this in the theorem's preamble, and we have relabelled the result as describing behaviour in a neighbourhood of a local minimum under those assumptions, which is the standard reading of the FedAvg bound we adapt [**?**].

Second, and more importantly: Corollary 2 in the submitted manuscript does not follow from Theorem **??**. The bound's heterogeneity term $\frac{2L}{\mu^2 T}(L + \Gamma_\alpha)$ applies identically to both architectures, and the spline-approximation residual $\mathcal{O}(G^{-(k+1)})$ is an *additional* error term for the KAN, not a mechanism that shields it from Γ_α. The partial derivative with respect to Γ_α is the same for both. The corollary therefore asserted the robustness conclusion rather than deriving it. Since our revised experiments no longer support that conclusion empirically either (R2.7), we have removed Corollary 2 rather than repair it, and the convergence analysis is now presented only as what it establishes: that the spline bias enters additively and does not scale with heterogeneity.

Where in the manuscript:

R2.2 Missing baselines: add communication-efficient FL methods (FedPAQ, Top-k SGD, or CG-FKAN); at minimum one compression baseline.

Response. Added FedPAQ with top-k sparsification at $k = 10\%$, with byte accounting that charges four bytes for each retained value and four for its index. At that budget the uplink falls from 7.69 to 1.64 MB for FedKAN-8, a factor of 4.7, but accuracy falls to 70.59% ($p = 0.003$) and for the MLP to 55.52% ($p < 0.001$). We report this as a negative result at this compression level rather than tune k until it flatters us: under Dirichlet $\alpha = 0.1$, where client updates already disagree, discarding 90% of each update compounds the disagreement. CG-FKAN is discussed in related work but not reproduced, since it requires a grid-sparsification schedule we could not reconstruct from the paper; we state that rather than approximate it.

We also note the accounting point, because it is easy to miss: SCAFFOLD doubles the uplink (15.38 against 7.69 MB) because each client returns a control variate as well as a model delta.

A comparison that charges every algorithm one model per round would make SCAFFOLD look free.

Where in the manuscript:

R2.3 Seed-17 pathology: provide class and feature distributions per client, why F-KAN-16 escapes the trap and MLP does not, and whether other seeds behave similarly.

Response. We measured the partition statistics the reviewer asks for, on all ten seeds so that seed 17 can be ranked rather than merely described, and the result is that *no statistic we measured singles it out.* On NF-ToN-IoT-v2 seed 17 ranks second of ten by client-size Gini (0.7235, behind seed 31 at 0.7600, which is not pathological), ninth of ten by mean per-client label entropy (seed 29 is lower still and performs normally), and its class-proportion spread of 1.0 is shared by eight of the ten seeds. Rank correlations against final accuracy are weak and not significant: Spearman +0.42 ($p = 0.23$) for entropy, −0.30 ($p = 0.40$) for Gini, +0.61 ($p = 0.06$) for smallest client size. We therefore report the failure mode as reproducible but unexplained rather than offer a mechanism the data does not support, and we have removed the sentence attributing it to an interaction between the partition and the class distribution.

Where in the manuscript:

R2.4 Hardware profiling: include preliminary measurements on Raspberry Pi or similar hardware; RTX 4090 times are not representative of IoT deployment.

Response. Answered under R1.5: single-thread CPU profiling with a conversion anchor, in place of the RTX 4090 numbers, which we agree answer a different question (centralised training throughput) and are labelled as such. We did not fabricate Raspberry Pi figures.

Where in the manuscript:

R2.5 Sensitivity analysis: vary grid size $G \in \{3, 5, 10\}$, spline order $k \in \{3, 4, 5\}$, and hidden width beyond 8 and 16.

Response. Added, on ten seeds each, with parameter counts alongside so that a capacity effect cannot be mistaken for a spline effect. Spline order is irrelevant here: k=4 gives +0.10 pp and k=5 gives −0.12 pp against k=3. Grid size matters and we re-ran it on ten seeds rather than the single run of the first version: G=3 costs 0.87 pp and G=10 gains 1.93 pp on NF-BoT-IoT-v2 ($p = 0.034$) and 3.89 pp on NF-ToN-IoT-v2 ($p = 0.095$), with the false-positive rate improving from 8.36 to 4.43% and from 21.29 to 16.01%. We should correct something here: an intermediate draft of this response cited a 84% false-positive rate for G=10, which came from the single run available at the time and is not representative. With ten seeds G=10 is better on every metric we report. It is still not free, adding 50% parameters, which breaks the parameter parity the comparison rests on, and roughly ten times the training time, and it does not repair the seed-17 partition, which moves from 51.79 to 56.12%. Hidden width is non-monotone and this surprised us: h=4, at half the parameters of our default, is 0.98 pp *better* than h=8, while h=16 and h=32 give +0.80 and +0.63 pp at two and four times the parameters. Within this family more capacity does not help, which is consistent with the regularisation reading of the spline basis and inconsistent with a capacity reading.

Where in the manuscript:

R2.6 Local convexity: provide empirical evidence (e.g. Hessian spectrum analysis) supporting the local convexity assumption.

Response. Measured, and the answer is unfavourable to the theorem. We ran Lanczos with full re-orthogonalisation on the exact Hessian-vector product at the converged federated solution (NF-BoT-IoT-v2, binary, 50 rounds, 8192 held-out samples, 40 Ritz directions). FedKAN-8 gives $\lambda_{\max} = 88.5$, $\lambda_{\min} = -0.134$, with 45% of Ritz values negative; the parameter-matched MLP gives $\lambda_{\max} = 0.52$, $\lambda_{\min} = -1.8 \times 10^{-3}$, with 27.5% negative. Negative curvature at the solution means the iterates settle at a saddle region, so strong convexity fails not only globally, as the reviewer notes, but locally as well. We state this in the revised theory section, and it is part of why we removed Corollary 2. We also note what the measurement does *not* show: the two condition numbers are the same order (662 against 285), so the Hessian spectrum does not explain any performance difference between the architectures.

Where in the manuscript:

R2.7 Hyperparameter justification: clarify whether Adam with $\eta = 10^{-2}$ was tuned per architecture, and whether KANs require different learning rates.

Response. This comment was correct and acting on it changed the paper's central claim. The submitted study used $\eta = 10^{-2}$ for every architecture. We swept six values spanning two and a half orders of magnitude, for all four architectures, on all ten seeds, on all four cells: 1,200 runs on one machine.

The parameter-matched MLP's best step size on NF-BoT-IoT-v2 is 10^{-1}, ten times the shared value. Giving it that value gains it 5.39 pp; the same freedom gains FedKAN-8 0.65 pp. The headline gap falls from +5.49 to +0.76 pp ($p = 0.74$ paired, bootstrap CI $[-3.39, +4.99]$). Across the four cells the advantage ranges from -0.49 to +6.23 pp with no consistent sign. The variance-reduction ratio behaves the same way: 2.53× becomes 1.05× once both sides are tuned, and is already 0.71×, 0.33× and 0.87× on the other cells.

Yes, KANs need a different learning rate from MLPs, and that turns out to be the paper's real finding rather than a nuisance: KAN is 2.7 to 5.4 times less sensitive to the choice. Section **??** is new, the title and abstract are rewritten around this, and the accuracy and variance claims of the first version are withdrawn.

Where in the manuscript:

R2.8 Seed-17 characterization: detail the client data distributions that make this partition uniquely challenging for KAN-8.

Response. Answered together with R2.3 above: the per-client distributions are now reported for all ten partitions, and they do not identify seed 17.

Where in the manuscript: